

AWS Project Overview

Project Components

- VPC and its components
- Auto Scaling Group (EC2)
- S3
- Load Balancer (WAF and ACM SSL Certificates)
- CDN
- IAM
- RDS
- Redis Cache
- EKS
- CloudWatch
- SNS

Environment

1. Development Environment

Purpose: Used by developers to write, build, and test code from scratch.

Key Activities:

- Code is developed, debugged, and unit tested.
- Integration with source control tools like GitHub or Bitbucket.
- Local testing with containers (e.g., Docker) or sandboxes.
- Developers configure environment variables, third-party APIs, and mock services.

Real-Time Use Cases:

- A new feature is implemented in the local development branch and tested using Postman or Swagger UI.
- Backend APIs are integrated with frontend modules and tested via mock data.
- Developers simulate various error conditions to handle exception flows.

2. QA (Quality Assurance) Environment

Purpose: Used by the QA team to test the application's functional and regression scenarios.

Key Activities:

- Functional, regression, smoke, and sanity testing.
- Manual and automated test suites executed.
- Bug tracking and verification of fixes.

Real-Time Use Cases:

- QA testers validate login functionality, form validations, business logic, and workflows.
- Selenium or Cypress test cases are triggered via Jenkins in QA.
- Bugs found are reported to the dev team using Jira.

3. Stage/UAT (User Acceptance Testing) Environment

Purpose: Used for client testing, performance evaluation, and business validations.

Key Activities:

- Load and performance testing with tools like JMeter, Gatling.
- Final review of features by product owners or clients.
- Dry-run of release candidate before production push.

Real-Time Use Cases:

- Client logs in to test new features like payment gateway integration or user onboarding flow.
- Performance engineers simulate 1,000 concurrent users to test system stability.
- End-to-end flows such as placing an order or generating reports are validated by stakeholders.

4. Pre-Production Environment (Staging Mirror)

Purpose: Replica of the production environment used to validate last-minute changes or hotfixes.

Key Activities:

- Final smoke testing of deployment packages.

- Verification of infrastructure-level changes (e.g., load balancer configurations, database migrations).
- Testing failover and backup strategies.

Real-Time Use Cases:

- A security hotfix is deployed to pre-prod and tested before applying to production.
- Validate SSL certificates, firewall rules, or DNS settings prior to production go-live.
- Final approval and release sign-off from DevOps or IT security team.

5. Production Environment

Purpose: The live, customer-facing environment where the actual application is accessed by end users.

Key Activities:

- Real-time application availability, performance monitoring.
- Incident and change management via ITIL processes.
- Logging, alerting, and continuous monitoring via tools like CloudWatch, Grafana, Prometheus.

Real-Time Use Cases:

- Users log in to purchase items, check reports, or access services 24/7.
- Monitoring detects a CPU spike; autoscaling is triggered to maintain availability.
- A zero-downtime deployment strategy (blue-green or canary) is used to release a new feature.

Project Execution Timeline with GUI creation.

1 Week - GUI Creation (Time-consuming task)

1 Week - Confluence Documentation

1 Week - Infrastructure and Application Testing and Sign-off

Terraform (Infrastructure as Code)

CLI Creation Benefits

Save time

Avoid human errors

Rapid infrastructure setup (within 1 hour)

Write test cases to verify application functionality

Infrastructure in IT

Infrastructure in IT refers to the components required to operate and manage IT environments, such as hardware (servers, routers, switches, modems), and software (like EC2, S3, VPC, etc.). When hosted on AWS, this is termed as cloud infrastructure. To automate GUI steps, Infrastructure as Code (IaC) tools like Terraform are used.

Terraform Overview

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp using Go.

IaC manages infrastructure through code files instead of manual configuration.

Resources include VMs, Elastic IPs, Security Groups, VPCs, etc.

Terraform uses HashiCorp Configuration Language (HCL) with .tf file extension.

Supports multiple providers (AWS, Azure, GCP, etc.).

Documentation: <https://www.terraform.io/docs/providers/index.html>

Pre-Requisites

Cloud Provider (AWS, GCP, Azure, etc.)

User credentials (Access Key and Secret Key via IAM user with admin policy)

Installation on EC2 (Amazon Linux)

Login as root user.

Run:

```
yum update -y
```

Install utilities:

```
sudo yum install -y yum-utils
```

Add repo:

```
sudo yum-config-manager --add-repo  
https://rpm.releases.hashicorp.com/AmazonLinux/hashicorp.repo
```

Install Terraform:

```
sudo yum -y install terraform
```

Check version:

```
terraform -v or terraform version
```

Help command: terraform -help

Terraform Commands

terraform init: Initializes a Terraform working directory.

terraform fmt: Formats Terraform configuration files to standard style.

terraform validate: Validates configuration syntax.

terraform plan: Generates and shows execution plan (dry run).

terraform apply: Applies changes to reach desired infrastructure state.

terraform destroy: Destroys Terraform-managed infrastructure.

First Terraform Script

```
provider "aws" {  
  region    = "ap-south-1"  
  access_key = "AKIAQ3EGWJG6Z2DT37RD"  
  secret_key = "C3lv9DA09mLvVh4GMZWUPiJkVwF31GXBSysD804"  
}
```

```
resource "aws_instance" "MyEC2Server" {  
  ami          = "ami-0cc9838aa7ab1dce7"  
  instance_type = "t2.micro"  
}
```

```
resource "aws_instance" "MyEC2Server" {  
  ami          = "ami-0cc9838aa7ab1dce7"  
  instance_type = "t2.micro"  
  key_name     = "Prometheus_Keys"  
  count        = "5"
```

```
}
```

Note

The general syntax for a Terraform resource is:

```
resource "<PROVIDER>_<TYPE>" "<NAME>" {  
  [CONFIG ...]  
}
```

PROVIDER: Cloud provider name (e.g., aws)

TYPE: Resource type (e.g., instance)

NAME: Identifier used in code

CONFIG: Specific parameters for the resource

Assignment

What is the default IaC tool for AWS?

List the advantages of using Infrastructure as Code (IaC).